# Patterned Agentic Comms Environment

## Introduction

An industrial-grade library of components for real-time and simulated real-time Ada applications.

**Index:** History | Infrastructure | Patterns | simulation | Portability | Prototyping | Reference | How-To | XP | Summary | API

**Key Points**

- **Purpose & Scope**
  - Presents an industrial-grade Ada library for real-time and simulated real-time applications, emphasizing hardware independence and open systems development.
  - Designed as a common simulation framework (CSF) to avoid proprietary infrastructures and languages.
- **Core Architecture**
  - Utilizes Ada's strengths for concurrency and runtime features, resulting in a compact, reusable component library (PACE).
  - Communication infrastructure is built on interprocess communication (IPC) using the command pattern.
- **Application Areas**
  - Deployed in architecture verification, conceptual modeling, robotic design, simulation training, requirements modeling, performance modeling, human factors, test environments, and timeline modeling.
  - Supports full-up simulations for trade studies, such as driving evaluations and collision detection.
- **Design Philosophy**
  - Focuses on simplicity and parsimony: "*A complex system that works derives from a simple system that works.*"
  - Ada coding patterns are preferred over language-neutral patterns (e.g., GoF), leveraging Ada's built-in features for prototyping and maintainability.
- **Key Patterns**
  - Command, Singleton, Rendezvous, External Stream, Proxy, Publish-Subscribe, Mailbox, Notify, Queue, Mutex, Pooled Resource, Wakeup (single/multiple/data/task), Channel, Buffered Command, Callback,

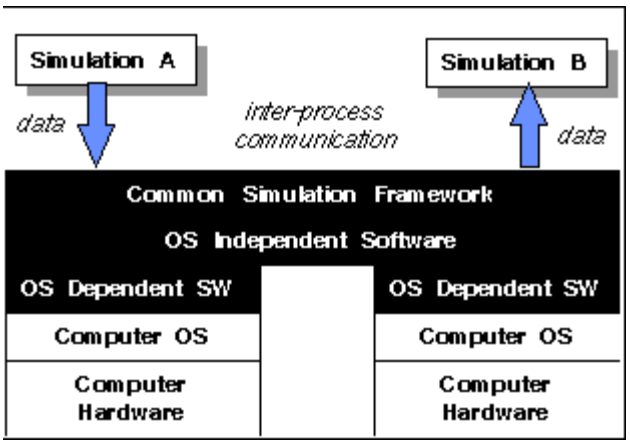
Persistent Command, Fault, Peek/Poke, Elaboration Order, Execute Once, Environment Variable, Configuration Data, Multicast, Shared Memory, 6DOF, CommandLine.
    - Patterns are designed to be concise, portable, and easy to implement in Ada.
- **Simulation Capabilities**
    - Built-in discrete event simulation engine enables timeline analysis, performance management, and design robustness checks.
    - Simulation mode is activated via environment variables; supports regression testing and debugging.
- **Portability**
    - 100% portable across Unix-like OSes; minor modifications needed for Windows.
    - GNAT Ada compiler recommended for cross-platform compatibility.
- **Prototyping Approach**
    - Advocates rapid prototyping using Petri Nets, state diagrams, and CRC exercises.
    - N-Tier architecture: User Interface, Middleware (Virtual Vehicle), Hardware Abstracted Layer (Plant Model).
    - Automated testing and instrumentation are integral to development.
- **Interprocess Communication (IPC)**
    - All IPC is message-based, using Ada's command pattern and logical node structuring.
    - Configuration files (nodes.pro, session.pro) define routing and launching of processes.
- **Extreme Programming & Development Practices**
    - Encourages continuous integration, minimal source management, and iterative development.
    - Stresses the importance of trace facilities, stack monitoring, and avoiding task priorities/timeouts in early prototyping.
- **Environment Variables**
    - Extensive use of environment variables for configuration, simulation control, and debugging.
- **Knowledgebase Integration**
    - Uses a Prolog-based inference engine (Gnu.Rule_Process) for flexible configuration and expert system capabilities.
- **Summary & Rationale**
    - Aims for device-independent, portable software suitable for evolving systems.

- Supports full lifecycle development from conceptualization to deployment, with competitive advantages in cost, quality, and time-to-market.

---

## Objective

This is an open systems development applying a computer hardware independent approach, originally intended for use as a common simulation framework (CSF):




The CSF approach avoids proprietary infrastructures and/or languages, with the use of Ada providing a standard for interfacing to other languages, via pragma statements.

The communication infrastructure uses interprocess communication (IPC) derived from the command pattern.

The library has been deployed on the following set of applications, in roughly chronological order:

1. Architecture verification model
2. Conceptualization or theory of operation model
3. Detailed robotic design models (and "what if?" experiments)
4. User's manual/training materials model, i.e. simulation trainer
5. Software requirements model
6. Performance modeling
7. Human factors model
8. Test environment model (definition of all potential instrumentation points)

9. Timeline model

Some of these capabilities have been combined so as to accomplish full-up simulations for conducting trade studies including: driving evaluations and robotic-assembly collision detection.

The path forward is to expand the horizon of uses and in particular, extend the requirements model into prototyping [Wood 92] and an executable architecture that can speed up the software development process.

---

# Infrastructure Path Forward

At its kernel is a set of library functions that perform discrete event handling. The result is a reusable component library -- the PetriNet Ada Component Environment. Note that the name is coincidentally similar to the ACE library for C++ [Schmidt 1994], which is another open-systems middleware library. Even though the capabilities of PACE match those of ACE, this library is significantly smaller (by over an order of magnitude), primarily through the use of the extensive concurrency and run-time aspects of Ada, which C++ lacks. The documentation includes the API (Programmer Manual in Annex A) which provides a simulation architecture guide.

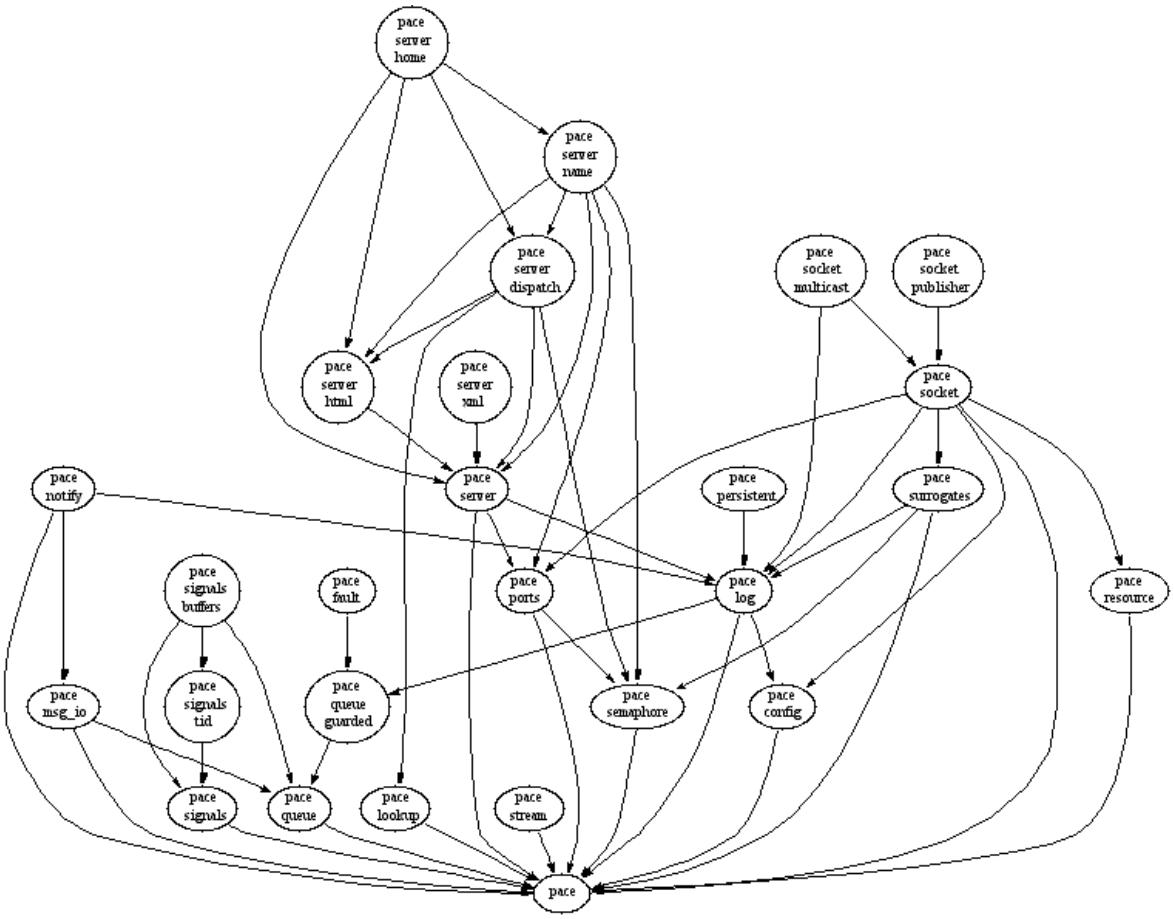
Apart from the kernel PACE libraries, several Ada and GNU reuse libraries have been incorporated during the development; these are recognizable by the appropriate prefix (either Ada, System, or Gnu). The current count for PACE is ~5700 SLOCs or 2600 semicolons.

As this report is a documentation of the PACE library, we should point out that simplicity can act as a double-edged sword. Often-times simplicity in implementation of a library places a burden on the application developer, who then potentially has to write more complex interface code to use the library effectively. On the other hand, making the interface too simple often leads to more code on the back-end to handle all the implicit requirements. We believe the PACE library succeeds on both sides: an easy-to-use application programming interface, together with very simple refactored patterns for application in the back-end. This follows the maxim of parsimony — *"A complex system that works derives from a simple system that works"*.

The basis behind the simplicity of expression was in finding Ada patterns that lend themselves to fast prototyping. If one keeps representations as simple as possible, it makes it that much easier to train coding practices, code inspections, and to simplify

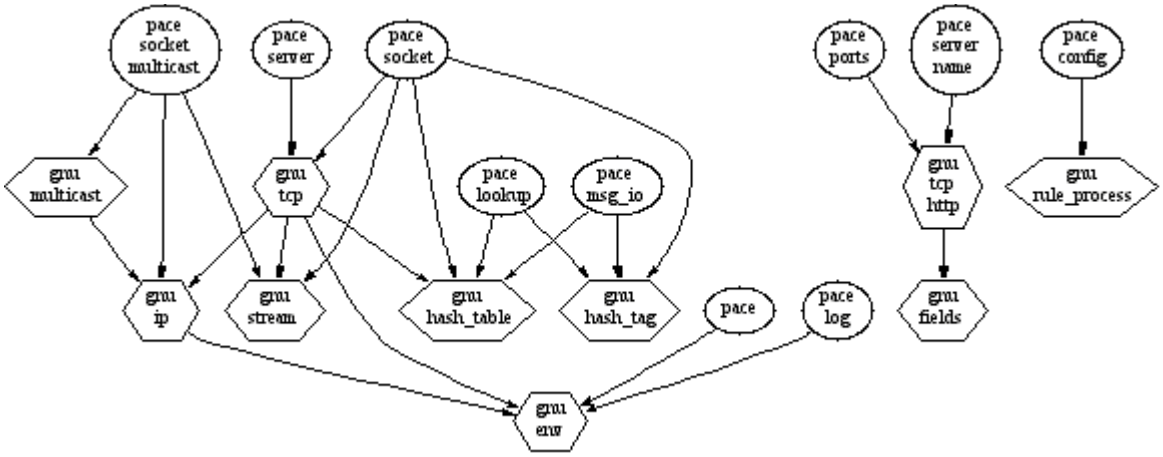
debugging of software during maintenance. That coincides with a goal of keeping developmental procedures as short as possible.

The PACE library consists of a set of packages that are rooted on a base type called `Msg`, which stands for message. The child packages follow patterns dependent on this base message type and the attributes that go with it. We will go over each of the packages individually, but to get an overall picture, the following figure illustrates the intra-dependency of the PACE library elements.




***PACE library intradependencies*** : The root class is the bottom layer, the most abstracted is at the top.

The GNU library consists of mainly operating system dependent services such as TCP/IP protocols and simple data structure algorithms such as hashes and parsers, and a substantial rule evaluator library based on a simple Prolog dialect.




***PACE library GNU dependencies***: The GNU libraries are indicated by hexagons, these typically are not used by application programs as they are abstracted via higher level PACE packages.

# Patterns

The PACE library is based on fundamental Ada coding patterns. This is in contrast to libraries that happen to use language-neutral patterns described in the so-called *GoF*[2] patterns book [Gamma 1994]. Unfortunately, since the *GoF* patterns do not consider Ada features, it is not too surprising that a *GoF* pattern yields a cumbersome solution in Ada[3].

What is a pattern? According to conventional wisdom, a "coding pattern" is a simple set of rules that provides for more compact notation and uniform representation when a certain behavior is repeated in the code *more than once*. Patterns by their definition are manually generated by humans at the keyboard. This is in distinction to a "coding wizard" which is capable of auto-generating code based on input by a GUI tool. Fortunately, all PACE patterns typically require at most a few lines of code, obviating the need for a wizard.

The Ada **protected type** programming idiom is an example of a "built-in" language pattern that can be obscured by an overly complicated O/S layer. Historically, there have been many occasions where developers have used semaphore O/S services instead of the much simpler Ada protected types. In general, the built-in kinds of Ada patterns are recommended for use in a PACE application as these are textbook referenceable.

The basic pattern is the so-called **command** pattern, which is a message-centric paradigm (hence the name `Msg` for the base class).

Another often overlooked pattern that is crucial to prototyping simulations is the **singleton** pattern. In software terms, a singleton describes an object that occurs only once in an executing program and typically has a lifetime that persists over the entirety of execution. This pattern effectively describes such abstracted objects as a robotic arm design, etc. For each abstraction, a vehicle (or some other product line) only contains a single instance of the object. Contrast this to other candidate non-singleton objects such as digital maps, operating system files, etc., of which there may be multiple instances of each class (each containing a handle which points to the appropriate object instance).

The **singleton** pattern is actually quite clumsy to program using C++ or Java classes, but is very straightforward using traditional Ada packages:

```
with Pace;
package Singleton_Object is
   pragma Elaborate_Body;
   -- The Ada pattern for a singleton: fill in the package with commands
end Singleton_Object;
```

The simplicity of the approach is actually an issue to some object-oriented purists who claim that it marks a regression to structured paradigms. Our response to that challenge is to consider that embedded software controlling hardware has different requirements than desktop and other application software. The goal is to create robust and maintainable code, not necessarily to worry about extensibility and reuse of hardware behaviors; such behaviors are difficult enough as it is to prove correct. A practical benefit of using singleton patterns is the capability of doing various white box tests on the executing software.

If the `Singleton_Object` encapsulates a task, it is considered to also be an *Active Object* or *Agent*, with the convention of naming each of these encapsulated tasks by the `Agent` identifier. A singleton without a task is just an Ada package with internal state.

A built-in Ada pattern is the tasking **rendezvous** which directly provides a synchronized transaction between two threads. Easy to measure that both the **rendezvous** and **protected type** patterns exhibit performance close to limits set by a real-time OS (such as LynxOS) context switch. In abstract terms it provides a canonical representations for any Petri Net interaction. For example, wherever a UML *synchronized* method call is indicated, a rendezvous should be (at least) considered. Interestingly, Ada compilers are quite stingy on resources when it comes to the rendezvous construct; through pooling of mutexes, a good compiler will generate only 1 or 2 mutexes for a task, independent of how may rendezvous entry points are declared for the task[4].

The final example of a pattern is probably the most important with respect to interfacing to an external system (such as a web browser). The Ada **external stream** pattern can be used anywhere a tagged type is defined. The fact that tagged objects can be externally represented by a character string header followed by a programmer defined data representation gives us great flexibility in implementing (1) **proxy** patterns on top of the **command** pattern, and (2) **url** patterns for interfacing to web applications. The built-in Ada dispatching runtime allows the **external stream** messages to get dispatched to the right **singleton** object.

If these last two pattern descriptions seemed overly dense and concise, well, that is a benefit of using collections of patterns *together*. If the appropriate Ada patterns are used in an integrated fashion, we can achieve large economies of scale without having to do much extra coding. This will become more clear as we step though each of the PACE patterns. It is also important to consider that conventional Ada programming techniques should be used where appropriate; this is in keeping with the philosophy that PACE is a middleware framework that ties together communications and concurrency philosophies. Note that if either the communication or concurrency implementations are too complex or slow, we don't have a chance at building the rest of the application in a rapid prototyping approach. That is why it is worthwhile to get these patterns correct and optimized for efficiency.

---

## Simulation

The built-in simulation capability of PACE sets the library apart from other middleware libraries such as ACE. A few hundred SLOCs enables a true discrete event simulation engine in PACE.

The simulation capability allows PACE to cover timeline analysis, performance management trades, and design robustness. For example, a timeline analysis uses simulated waits to solve the problem of accurately predicting end-to-end scenarios. Performance management drills down at a finer level for fine-tuning scheduling and loading and providing software budgets. Not as well known, but critically important is the capability of simulated time to check robustness of designs — since the simulation engine can run at hyper-real-time, it is able to detect logic flaws, deadlock, and race conditions in short order [5].

In general, the software community at large has not appreciated a need for such a critical analysis of designs, probably due to their over-reliance on *ad hoc* procedures (such as UML and before that Rational Rose) which concentrate on static relationship checks. However, the fundamental capability provided by dynamic simulation

matches that required for proving out digital designs (via the discrete event simulation engine built in to the hardware description language VHDL) and for proving out controller algorithms (via the discrete event engine built in to Matlab). This fact is something that digital HW design engineers learned long ago, and is key to recouping huge financial investments in production runs where the end-product has to work the first time out. We advocate that tactical software development will benefit from this same early commitment to simulation.

Having been convinced that success depends on a simulation capability, you may next question if the upfront effort is too difficult. This is actually not at all the case since the application requires only simple primitives to get going. The patterns `Wait` and `Wait_Until` are one-liners in application code. A by-product of incorporating a discrete event simulation engine into the code allows one to do accurate timing *regression* tests and compare against reference timeline models[6].

**Development rules to allow discrete event simulation mode.** We have a few straightforward rules:

1. Do not use the built-in Ada `delay` in code statements, instead use `Pace.Log.Wait` statements.
2. Every agent task should be registered via `Pace.Log.Agent_Id (ID)` statements (the ID follows the Unit ID pattern).
3. The main procedure registers via `Pace.Agent_Id` in its first statement.

The wait statements are only triggered after the main procedure is elaborated so no background tasks should be idling on a wait delay until the main procedure is kicked off. The fundamental rule is not to invoke any significant behaviors until the main procedure is elaborated. To ease debugging, every task should also have a handler implemented `"exception when E : others => Pace.Log.Ex (E);"` to keep track of potential exceptions that may be raised.

---

# Portability

The PACE library is 100% portable across Unix-like operating systems (SGI IRIX, Sun Solaris, PowerPC, LynxOS, PC Linux). It requires a few modifications in the `Gnu.IP` package layer for PC Microsoft Windows development [7]. The recommended Ada compiler to use is GNAT since it is mature and available on most every 32-bit computer platform known. In practice, GNAT has become a commodity product much like all the standard UNIX utilities, since it is currently included in the current GCC cross-compiler collection.

There have been a few issues related to lessons learned on computing platform:

1. Watch out for UNIX signals, especially with respect to multi-tasking. Unhandled signals such as `SIGPIPE` may shutdown programs if they are not handled properly.
2. Try not to rely on Unix services such as IPC message queuing, semaphores, dynamic/shared libraries, and shared memory as well as UNIX signals to keep compatibility with MS-Windows.

---

# Prototyping

For the initial design of a prototype, we have found it best to sketch a few diagrams (Petri Nets and/or State Diagrams) and use index cards during rudimentary CRC exercises to fill in object behaviors. The active objects contain the state and an Ada task (thread) which controls the cyclic flow and state transitions. A good reference for how to think in terms of active objects is [Sanden 1999]. Sanden's general principle is that software entities generally have life-cycles which are conveniently described by Ada tasks. He goes on to describe three supporting principles:

**Entity-Life Modeling of Concurrent Software**

- Design is not a separate effort divorced from the implementation.
- The design deals with how to use the means of expression available to the programmer.
- Design is a direct bridge between the problem domain and the execution domain.

A typical vehicle simulation consisted of 180 tasks which could be run in either discrete event simulation mode or real-time mode quite easily. Maintaining an entire vehicle (or 2) simulation in a single executable may seem computer intensive, but is certainly possible and should be a requirement given the use of lightweight patterns. The later repartitioning of execution across multiple processes and processors is that much easier if the initial simulation provides a single executable reference (the semantic equivalence of the two implementations - distributed and local - is described in the Ada 95 Distributed Systems Annex as Reference Note E.1).

Static modeling has a place in prototyping but we follow the suggestion of Grady Booch, who has this to say about the need for dynamics in a model [Booch 1996]:

**Static Versus Dynamic Semantics**

The four diagrams we have introduced thus far are largely static. However, events happen dynamically in all software-intensive systems: objects are created and destroyed, objects send messages to one another in an orderly fashion, and in some systems, external events trigger operations upon certain objects. Not surprisingly, describing a dynamic event in a static medium such as a sheet of paper is a difficult problem, but it confronts virtually every scientific discipline.

We agree with proponents of Extreme Programming who say: (1) use the back of the envelope or napkin to sketch out early ideas, and (2) use the early prototype to generate artifacts for design review. This is probably the best approach to create something akin to the *Executable Architecture* [9].

The basic prototype for a vehicle simulation has followed an N-Tier approach as shown below[10].

| | | |
|---|---|---|
| **1** | **User Interface** | T E S T |
| **2** | **Middleware => The Virtual Vehicle** | |
| **3** | **Hardware Abstracted Layer => The Plant Model** | |

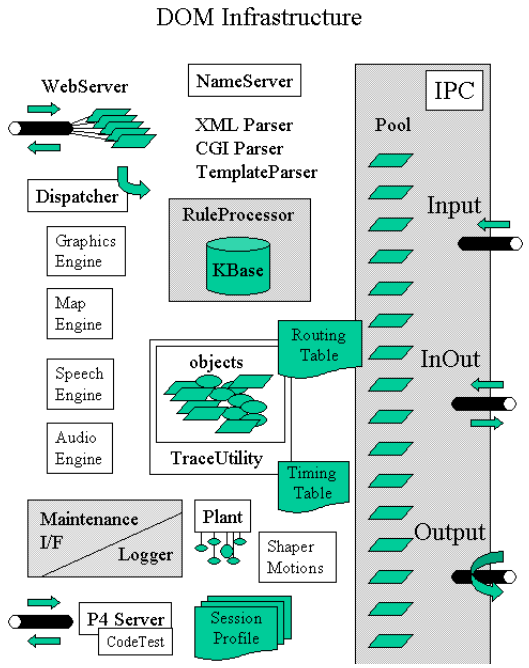
1. **User Interface.** The most technologically notable GUI is the Web interface, because it can be run on *any* device that can run a Web browser (virtually any workstation, PC, PDA, Cell Phone, etc.)
2. **Middleware / Virtual Vehicle.** The term "virtual vehicle" connotes the separation of concerns between abstracted vehicle behaviors and the underlying physical mechanisms.
3. **Hardware Abstracted Layer** The lowest layer does not have to be hardware, it can be anything – e.g. a real time database, an interface to a war-gaming simulation. However, if it is hardware, for simulation purposes, this becomes a plant model.

At each layer, automated tests are executed to maintain a solid, robust base of code. Especially pertinent is the utility of automated testing for executing scenarios. The P4 (Peek/Poke/Performance Profiling) library and tools were used to this end on both simulation and later on tactical code.

Stressing the importance of testing, it stands to reason that the first line of tactical code that shall be written will be a trace/instrumentation statement. From this adherence to a "Design for Test" strategy, we can automatically generate:

1. Calling trees
2. Directed graphs
3. Swim-lane diagrams
4. Detailed timeline information including:
    - Start & stop times
    - Synchronization points
    - Concurrency

A typical simulation application is structured as shown below. This is drawn from an "infrastructure-centric" perspective of a dynamic object model (DOM). The main application components are seen in the center as a cluster of passive objects (ovals) and active objects (tasks). The majority of these objects are singletons, although utility classes and instanced classes are also used where appropriate. All singletons are traced. There are also plant components which are typically associated with the tactical objects, hanging off the objects as "leaf" nodes but logically grouped together so as to share "ground truth" information.




The objects communicate via the command pattern. If the objects are distributed, the pool of IPC tasks on the right side handles the communication with the destination rules determined by the routing table (i.e. kbase). A multi-tasking web server on the

upper left provides dispatching through the URL command pattern; parsers for XML, CGI, and Template/ASP decoding can used at this point.

At the bottom, we also add the P4 server task to control launching and interactive debugging/testing. The session profile determines where executables reside.

The external tools attached to the DOM simulation are shown in the smaller boxes. These are typically engines to do graphics and other sensory input/output.

We expect to continue to follow this general structure as PACE is used in new application areas.

---

# PACE Reference

The PACE Reference section describes the PACE development patterns. For example, this is where we describe the abstracting of interprocess communication (IPC) in terms of the command pattern in greater detail. The most productive way to follow patterns is to look at code already developed and mimic that approach (hence the term patterns).

| | | |
|---|---|---|
| **"Command" Pattern** | `with Pace;` | Context Clause |
| | The fundamental Command Pattern enables ... | Description |
| | `type My_Msg is new Pace.Msg with ... record;`<br>`procedure Input (Obj : in My_Msg);` | Example of use |
| | Place in a package specification. Use Input, Inout, or Output depending on data flow | Instructions |
| **Pattern Template (using the "Command" Pattern as an example)** | | |

Following a template described in the table above, we will document the current patterns used for prototyping an architecture. These patterns are specifically Ada 95 and are portable to the extent that they have no dependence on the underlying operating system (specialized patterns that are used to provide interfaces to the external OS or to other tools are listed at the end). The patterns are both portable and concise because they use the built-in features of the language, such as tagged types, protected types, generics, and tasking. Only a few of the patterns take more than a few lines of code to setup, indicating a good match to the design intent of the language.

The various PACE patterns are listed below:

| | | | | | |
|---|---|---|---|---|---|
| Agent | Buffered | Callback | Channel | Command | CommandLine |
| Configuration | Data | Discrete | Dispatched | Elaboration | Environment |
| Execute | Expect | External | Fault | Log | Mailbox |
| Message | Multicast | Multiple | Mutex | Notify | Peek/Poke |
| Persistent | Pooled | Proxy | Publish | Queue | Shared |
| Single | Surrogate | Synchronized | Tag | Task | Trace |
| URL | Unit | 6DOF | | | |

If we were to define a paradigm for how the patterns typically fit together, a passable description may be to call the end result a "class-based messaging architecture". The PACE infrastructure uses many variations on the class-based command pattern. Such an essential pattern provides benefits in (1) instrumenting code and (2) allowing for other building block patterns to be constructed. The PACE implementation combines inheritance and class-wide operations on the base message classes. The base classes contain time stamps, task IDs, and other identifiers which can be used to trace execution. This is the basic architectural pattern that was instituted at the start of the modeling effort, intended to support both discrete event simulation and real-time simulation. Other patterns evolved based on needs of simulating the software design.

## Command (or Message) Pattern

```
with Pace;
```

Command messages are declared in a package specification, with at least one of the 3 available primitive dispatching operations also specified. The package body requires the filled-out operational code for the selected primitives.

```
   type Msg_Name is new Pace.Msg with ...;    -- defined in
Msg_Name_Package
   procedure Input (Obj : in     Msg_Name);  -- optional primitive #1
   procedure Inout (Obj : in out Msg_Name);  -- optional primitive #2
   procedure Output(Obj :    out Msg_Name);  -- optional primitive #3
```

In formal terms, the automatically inherited operations include `Input`, `Output`, and `Inout`. Depending on the parameter mode desired (whether data is "in", "out" or "in out"), one or more of these operations can be declared for each subclass'ed command `Msg`. The *command* pattern is also mentioned in the GoF patterns book, but the Ada implementation differs substantially, particularly in its lack of pointer manipulation.

On the client side, instancing of command messages is straightforward.

```
   declare
```

```
      use Msg_Name_Package;
      Msg : Msg_Name;
   begin
      Input (Msg);
   end;
```

In many cases, a "class-wide" procedure declaration is used in conjunction with a command message. The built-in class-wide to primitive operation dispatching of Ada allows further bulding block utilities to be created from the command pattern. These utilities include such patterns as Trace and Proxy.

## Dispatched Command Pattern

```
with Pace;
```

This pattern is a slight variation of the client-side invocation of the command pattern. Instead of calling the primitive operation directly, a class-wide operation is used which effectively enables redispatching to the appropriate primitive. The target of the indirection can be changed by registering a different class-wide callback.

```
   declare
      Msg : Msg_Name_Package.Msg_Name;
   begin
      Pace.Dispatching.Input (Msg);
   end;
```

Note that these first 2 patterns are the essence of the architectural paradigm.

## Discrete Event Simulation Pattern

```
with Pace.Log;
```

The two PACE timers (relative delay and absolute delay) can be switched to a discrete event simulation mode by setting the environment variable to PACE_SIM=1

```
   declare
      Time : Duration := Pace.Now;
   begin
      Pace.Log.Wait_Until (Time + 100.0);  -- Wait 100.0 s after the
clock time
      Pace.Log.Wait (10.0);                -- Wait 10.0 s from
invocation
   end;
```

The call `Pace.Now` returns the current time in seconds from the start of the simulation. For absolute wait, call `Wait_Until` with time in seconds (starting from start of sim). For relative wait, call `Wait` with time in seconds.

If the need arises to set up a timer on a thread, use the `Timer_Start` and `Timer_Expired` services. Like the `Wait` services these are guaranteed thread reentrant safe (since they use the built-in `Ada Task_Attribute` library;

```
procedure Timer is
   Expired : Boolean;
begin
   Pace.Log.Agent_ID;
   Pace.Log.Timer_Start (9.0);
   Pace.Log.Wait (10.0);
   Expired := Pace.Log.Timer_Expired;
   -- Post-Condition: Expired = TRUE
end Timer;
```

## Trace (or Instrumentation) Pattern

```
with Pace.Log;
```

The library contains class-wide utilities which apply to any derivation of `Pace.Msg`. One of these, the class-wide operation called `Pace.Log.Trace` provides a convenient way to instrument the code, as it accepts instances of `Pace.Msg'Class` as its parameter.

```
procedure Input (Obj : in Msg_Name) is
begin
   Pace.Log.Trace (Obj);  -- always put this at the end of the
procedure!
end Input;
```

On trace output synchronization semantics are: **=>** simple, **>>** synch, **->** asynch, **<>** release

## Unit Identification Pattern

```
with Pace.Log;
```

This pattern allows one to declare a character string identifier which automatically matches the enclosing module name. This is typically declared in a package body.

```
function ID is new Pace.Log.Unit_ID;
```

In general, placing character strings in code to identify packages is a poor idea; for example if maintenance occurs the strings may need to be updated. This provides an automated approach.

Along the same lines, PACE uses object tags in hash tables to textually associate data with messages. This can be demonstrated by combining the Tag Identification Pattern and Message Lookup Patterns.

### Agent (or Task) Identification Pattern

```
with Pace.Log;
```

This pattern allows one to register the name of a task. Use this with the Unit Identification pattern. All tasks are released when the default `Pace.Log.Agent_ID` is called from the main procedure.

```
function ID is new Pace.Log.Unit_ID;

Storage : constant Integer := Pace.Log.Task_Storage (ID);

task Agent is
   pragma Storage_Size (Storage);
end Agent;

task body Agent is
begin
   Pace.Log.Agent_ID (ID);
   ...
```

### Tag Identification Pattern

```
with Pace;
```

If the need arises to get the character string representation of a command message, use the following pattern:

```
procedure Input (Obj : in Msg_Name) is
begin
   Pace.Log.Put_Line (Pace.Tag (Obj) & " called");
end Input;
```

### Log Exception Pattern

```
with Pace.Log;
```

Log a descriptive string without blocking, allowing a concurrent task to monitor. In practice, each task exception handler calls `Pace.Log.Ex` with an optional text string. The monitoring task needs to call the function `Pace.Log.Wait_For_Ex` to retrieve strings placed on the queue.

```
exception when E : others =>
```

```
Pace.Log.Ex (E, "extra info");
```

In practice it is useful to log all exceptions, and make sure there is an exception handler in every task. A task that mysteriously exits due to an unhandled exception is often hard to diagnose, and using heartbeat signals to detect the absence of a task is problematic for non-polled tasks.

## Expect Pattern

```
with Pace.Log, Ses.PP;
```

Send a descriptive string to standard IO. This is used with the SES P4 test utilities to detect occurence of a unique string on the output.

```
Pace.Log.Put_Line ("string");
```

To turn this debugging off or for redirecting IO, use the `Set_Display` service,

```
procedure Null_Put_Line (Text : in String) is
begin
   null;
end;
...
Pace.Log.Set_Display (Null_Put_Line'Access);
```

## Synchronized Message Passing (Rendezvous) Pattern

```
with Pace;
```

If the implementation of `Input` makes a call to a task rendezvous, the transaction is **synchronized**.

```
task Agent is
   entry Input (Obj : in Msg_Name);
end Agent;

-- Agent Task
--
task body Agent is
begin
   accept Input (Obj : in Msg_Name) do
      Pace.Log.Trace (Obj);
   end Input;
   ...
end Agent;

-- Client Task
--
procedure Input (Obj : in Msg_Name) is
begin
   Agent.Input (Obj);  -- Transfer control and data to Agent
```

```
end Input;
```

The calling Client must not be the same task context as the Agent, otherwise the execution will deadlock. In practice, this does not cause a problem because it is very easy to detect deadlock through either code inspections or executable tests.

## Surrogate (Asynchronous) Message Passing Pattern

```
with Pace.Surrogates;
```

This pattern uses a surrogate task to emulate an asynchronous message passing protocol, this is also often referred to as "send and forget" semantics. If a dispatching operation called `Input` is declared, then the Asynchronous send pattern can be applied. There are two flavors to this pattern, one that uses a dedicated task and one that uses a pool of surrogate tasks. Both approaches call the dispatching operation named `Input`. Note that since the asynchronous protocol does not require a return, neither `Output` and `Inout` calls are applicable since they do expect data back from the called primitive.

The easiest flavor of the pattern to use is the pooled variation. One task in the task pool calls the class-wide operation as determined in the `Pace.Surrogates` package.

```
declare
   Msg : Msg_Name;
begin
   Pace.Surrogates.Input (Msg);
end;
```

The other variation is to use a dedicated task to manage the handoff from the client to the surrogate, who can then asynchronously deliver the message.

```
package Async is new Pace.Surrogates.Asynchronous (Msg_Name);
...
declare
   Msg : Msg_Name;
begin
   Async.Surrogate.Input (Msg);
end;
```

The recommendation is to always use the pooled version pattern since it requires less code. An environment variable PACE_MAX_SURROGATES controls the number of tasks in the pool.

## Mailbox Pattern

```
with Pace.Msg_IO;
```

This pattern pairs up sending and receiver tasks via a command message. Application developers typically won't use this pattern directly this but it is needed by the Notify pattern. It is thus more of a building block pattern for use by other Pace abstractions.

```
   package Mailbox is new Pace.Msg_IO (479); -- prime number for hash
table size

   -- Server side
   Msg : Msg_Name;
   Mailbox.Await (Msg);

   -- Client side
   Msg : Msg_Name;
   Mailbox.Send (Msg);
```

## Notify Pattern

```
with Pace.Notify;
```

This simple but very powerful pattern provides a trigger/suspend pair on a message. The message has two pre-built operations (`Input` and `Inout`) built up from the Mailbox pattern. The inherited `Inout`primitive will suspend on the subscription message, while the inherited `Input` provides a built-in trigger (or publish) mechanism from a concurrently executing task.

```
type Msg_Name is new Pace.Notify.Subscription with
   record
      Data_Field : ...
   end record;

-- Server-task suspend
task body Agent is
   Msg : Msg_Name;
begin
   Inout (Msg);
   Msg.Data_Field := ...

-- Client-task trigger
declare
   Msg : Msg_Name;
begin
   Msg.Data_Field := ...
   Input (Msg);
```

In a pinch this allows the task rendezvous mechanism to be extended to regions outside the task body.

## Queue and Guarded Queue Pattern

```
with Pace.Queue.Guarded;
```

The Guarded Queue pattern provides a re-entrant safe mechanism to pass constrained data.

```
package Q is new Pace.Queue (Item_Type);
package Guarded is new Q.Guarded;

Guarded.Get (Item);   -- Task #1

Guarded.Put (Item);   -- Task #2
```

The queue operations `Guarded.Put` and `Guarded.Get` are available. Note that since the queue generic requires constrained types, the `Pace.Msg'Class` cannot be used directly as an *Item_Type*; instead use the `Pace.Channel` type here (see Channel Command pattern).

## Mutex Pattern

```
with Pace.Semaphore;
```

This pattern protects data from simultaneous access via an automatically scoped semaphore object. This is a fairly common textbook Ada pattern.

```
use Pace.Semaphore;
My_Mutex : aliased Mutex;
...
declare  --
   L : Lock (My_Mutex'Access);
begin
   -- Task #1 access locked data until end of scope
end;
...
declare  --
   L : Lock (My_Mutex'Access);
begin
   -- Task #2 access locked data until end of scope
end;
```

## Pooled Resource Pattern

```
with Pace.Resource;
```

This pattern provides a pool of guarded keys that can be used to access task or re-entrant critical data. Currently, it is used as a building block pattern for other PACE abstractions, such as the `Pace.Socket`package.

```
type Pool_Range is range 1 .. 10;
package Pool is new Pace.Resource (Pool_Range);
```

```
declare
   Key : Pool_Range;
begin
   Key := Pool.Get;
   -- access locked data until end of scope
   Pool.Free (Key);
exception
   when Pool.Resource_Error =>
      Pool.Free (Key);
end;
```

## Single Event Wakeup Pattern

```
with Pace.Signals;
```

Instance a `Pace.Signals.Event` protected type in a package body that has access to multiple threads of control.

```
Evt : Pace.Signals.Event;
...
Evt.Suspend;  -- Task #1
...
Evt.Signal;   -- Task #2  which wakes up suspended Task #2
```

## Multiple Event Wakeup Pattern

```
with Pace.Signals;
```

Instance a generic `Pace.Signals.Multiple` in a package body that has access to multiple threads of control. An enumerated type or other ranged scalar type can be used to specify the desired signalling states.

```
type Colors is (Black, Red, Blue);
package Evts is new Pace.Signals.Multiple (Colors);
```

Three variations of multiple signal control exist. The first specifies waiting on a <u>specific</u> enumeration value:

```
Evts.Await (Black);    -- Task #1
...
Evts.Signal (Black);   -- Task #2  wakes up suspended Task
#1
```

The second describes waiting on <u>any</u> enumeration:

```
declare
  Color : Colors;
begin
  Evts.Await_Any (Color);  -- Task #1
  -- post-condition ->
  --  Color returns Black
end
...
Evts.Signal (Black);       -- Task #2  wakes up suspended
Task #1
```

The last variation describes waiting on all enumerations:

```
   Evts.Await_All;        -- Task #1
   ...
   Evts.Signal (Red);     -- Task #X
   ...
   Evts.Signal (Blue);    -- Task #Y
   ...
   Evts.Signal (Black);   -- Task #Z  which finally wakes
suspended Task #1
```

## Data Wakeup Pattern

```
with Pace.Signals;
```

This is another pattern to use instead of a rendezvous (although I would typically use a rendezvous because of type-safety and simplicity considerations). To use this pattern, first instance a `Pace.Signals.Shared_Data` protected type in a package body that has access to multiple threads of control.

```
   Obj : aliased Msg_Name;
   ...
   Data : Pace.Signals.Shared_Data (Obj'Access);

   task body Agent is         -- Task #1
      Msg : Msg_Name;
   begin
      Data.Read (Msg);
   exception
      when Pace.Signals.Data_Mismatch =>
         -- Uh-oh, types don't match
   end;

   declare                    -- Task #2  wake up Read by
calling Write
      Msg : Msg_Name;
   begin
      Data.Write (Msg);
   end;
```

Of all the patterns defined so far, this one has the potential for data type mismatches. If the reader waits on a message that doesn't match the type of the writer, an exception will be raised. Compare this to the Synchronized Message Passing or Notify pattern and you can see that the extra complexity of the Shared Data Wakeup pattern makes the type-safe rendezvous or notify a much better choice.

## Task Wakeup Pattern

```
with Pace.Signals.Tid;
```

This pattern uses built-in Ada task identifiers to associate waiting tasks with triggering clients (TID stands for Task ID).

```
   Id : Pace.Thread;             -- Static variable
   ...
   task body Agent is            -- Task #1
   begin
      Id := Pace.Current;
      Pace.Signals.Tid.Wait;
   ...
   Pace.Signals.Tid.Signal (Id); -- Task #2  wakes up
suspended Task #1
```

The Agent Wakeup pattern is used in the `Pace.Signals.Buffers` service. Each `Msg'Class` contains a TID field which is used to pass thread identifiers between active objects.

## Channel (Unconstrained Command) Pattern

```
with Pace;
```

This pattern allows heterogeneous and unconstrained command messages to be mixed in a safe and controlled way. The covenience operator "+" is defined to allow simple construction of the "channel" messages.

```
   use Pace;
   Msg : Msg_Name;
   Chan : Pace.Channel_Msg := +Msg;
   ...
   Input (+Chan);  -- Dispatches to primitive operator
```

The Channel pattern is used in the `Pace.Signals.Buffers` service and in the command callback pattern.

## Buffered Command Pattern

```
with Pace.Signals.Buffers;
```

This pattern uses the Channel pattern and the Agent Wakeup pattern to enable synchronized passing of heterogeneous/unconstrained data. This is accomplished by buffering messages in a protected queued data structure.

```
   type Msg_Name is new Pace.Msg with
       record
           Char : Character;
       end record;

   Queue : Pace.Signals.Buffers.Buffer; -- Contains Class-wide Queue
```

```
task body Agent is              --
    Item : Pace.Channel_Msg;  -- Single Class-wide element
    use type Pace.Channel_Msg;
begin
    Pace.Signals.Buffers.Get (Queue, Item);
    -- Convert from class-wide to specific type
    Pace.Log.Put_Line ("Char = " & Msg_Name (+Item).Char);

...

declare  -- other task
    Msg : Msg_Name;
begin
    Msg.Char := 'c';
    Pace.Signals.Buffers.Put (Queue, Msg);
```

The Buffered Command pattern is useful for composing Command messages out of other Command messages, where the components can be heterogeneously defined. On the receiving end, the individual channeled components can be dereferenced and then polymorphically dispatched to their primitive command operation.

## Proxy (Socket) Pattern

```
with Pace.Socket;
```

This simple pattern creates a Proxy to enable message sending via a socket[11]. The interface is structurally similar to the pooled asynchronous call, but the class-wide operation redirects to the socket IPC protocol instead before dispatching at the remote site.

```
   Msg : Msg_Name;
   ...
   Pace.Socket.Send (Msg);  -- use the Ack flag if synching is
important
```

The protocol is two-way if the `Send_Inout` or `Send_Out` is used and either synchronous or asynchronous if the `Send` operation is used.

Run-time configuration is provided by the "nodes.pro" file, configurable by the PACE_NODES environment variable. See the separate section on this topic.

Good use of patterns eliminates the need for "wizard" code-generation algorithms. Whereas marshalling of data is a difficult problem, a command-based proxy pattern is simple to implement and simple to enforce via coding guidelines.

## Publish-Subscribe (Asynchronous Notify) Pattern

```
with Pace.Socket.Publisher;
```

Publish-Subscribe is effectively a second-order pattern that requires an extra level of protocol on top of the basic command Input pattern. The subscription protocol is much like the Notify pattern; however an intermediate step for maintaining a subscription list is required. To identify the protocol, we subclass a `Subscription` message from `Pace.Msg`.

```
package Status_Pkg ..
  type Status is new Pace.Msg with
     record
        Data : ...;
     end record;
  procedure Input (Obj : in Status);
```

In the body, implement the subscription list:

```
   List : Pace.Socket.Publisher.Subscription_List (Max_Subs);  -- Max
defaults to 1

   procedure Input (Obj : in Status) is
   begin
      Pace.Publisher.Subscribe (List, Obj);
   end Input;

   task body Agent is
      Local_Status : Status;
   begin
      ...
      Local_Status := ... -- update the locally persistent state
information.
      Pace.Publisher.Publish (List, Local_Status);
```

On the client-side, we must subclass from the server message. This can be done in the package body:

```
   type My_Status is new Status with null record;
   procedure Input (Obj : in My_Status);
   procedure Input (Obj : in My_Status) is
   begin
      -- Grab the data
   end;

   -- To subscribe, create instance of My_Status, but (important!)
convert to Status
   declare
      Msg : My_Status;
   begin
      Status_Pkg.Input (Status (Msg));  -- Locally subscribe
```

Only one task is explicitly involved in this pattern, since in the local Agent the task itself is responsible for invoking the callback Command message.

**To subscribe the client remotely**, replace within the declare block the following:

```
declare
   procedure Subscribe is new Pace.Socket.Observer
       (Remote => Status_Pkg.Status, -- Needed for remote IPC
        Local  => My_Status);
begin
   Subscribe; -- Remotely subscribe
```

The communication pattern is reliable in that it uses TCP sockets when interprocess exchange occurs. Other than that, use this pattern with careful justification in that it can be prone to race conditions if used excessively. The worst-case example of this condition is where every object publishes data asynchronously but there is no central level of synchronized control. The giveaway for this predicament is the liberal use of delay statements to try to mitigate race conditions.

The publish-subscribe pattern is probably oversold in terms of use. A typical DOM has predominantly used the proxy pattern for IPC. The actual frequency of publish-subscribe pattern usage is fairly low. As an alternative, blocking of command pattern events ala the Notify pattern is an easy implementation decision and has found more widespread usage. For example, converting Notify to Proxy is trivial.

## URL Command Pattern

```
with Pace.Server.Dispatch;
```

Sending a text command to the PACE Web Server uses the Ada tagged type semantics in a novel way to do transparent dispatching. If ASCII encoded data is also attached to the text (i.e. URL) message, the protocol pattern is also known in current web circles as CGI or XML RPC (remote procedure call) or SOAP (simple object accesss protocol) programming. It becomes very simple to implement in Ada.

```
   package body Pkg is
      -- Declare a subclass derived from Pace.Server.Dispatch.Action
      type Msg_Stimulus is new Pace.Server.Dispatch.Action with null
record;
      procedure Inout (Obj : in out Msg_Stimulus);
```

```
   use Pace.Server.Dispatch;
   procedure Inout (Obj : in out Msg_Stimulus) is
   begin
      Pace.Log.Put_Line ("Stimulus received : " & +Obj.Set);
   end Inout;

-- Register through the Save_Action call, either in a task or at the
-- package elaboration section
begin
   Save_Action (Msg_Stimulus'(Pace.Msg with Set => +"OK"));
end Pkg;
```

The internal web server keeps track of all registered action requests. A typical URL Action Request lloks like:

```
http://wcss239:5601/PKG.MSG_STIMULUS?set=OK
```

The bolded part is the salient PACE interpreted URL. Note that whatever package that the `Msg_Stimulus` dispatching primitive is defined in, this name has to prepended to the URL. In so doing, the pattern becomes extremely easy to maintain.

The URL pattern allows a backdoor interface to allow the injection of external sensor messages into the executable. Variations have been used with the VAPS tool. In some programming circles, the `Save_Action` call is part of a factory or a registration pattern, since these objects are created in a factory line fashion, each one stamped out prior to use. Note that the command pattern in general has no such restriction, because by definition they have been registered prior to use by the Ada run-time. However, when external messages are considered, the run-time has had no chance to create an object instance before the external tag arrives; thus the requirement for the registration process. Moreover, providing this registration allows us to set defaults, as in the `"OK"` example above.

If the package bodies have no spec interface, somewhere in the main the package must be "withed". This is enough to enable the appropriate action dispatching for the executable. The actual instancing of the web server is described elsewhere; the server instancing doesn't count as a pattern because it is instantiated at the main driver level.

## Message Lookup (Tag Hash) Pattern

`with Pace.Lookup`

Provides a lookup (i.e. hash or set) table to `Msg` tags. The internal tag of a message is convertible to an external tag (human readable) through

the `Ada.Tags` utilities. Although one can instance this pattern directly, it is used more often in other pattern utilities than on its own.

```
-- Instance a Lookup table with hash size.
package Hash is new Pace.Lookup (479); -- Prime number is best

Obj : Msg_Name;

-- set a value in Lookup table corresponding to a message tag
Hash.Table.Set (Msg_Name'Class'Tag, +Obj);

-- get value and dispatch
Pace.Dispatching.Input (+Hash.Table.Get (Msg_Name'Class'Tag));
```

Note that this uses the Channel Command pattern which manages the memory automatically. Use a Mutex pattern if more than one task will access this; see `Pace.Server.Dispatch` for a complete example.

Don't worry if the hash size seems like it won't contain all values; it will actually spill over and start growing to accomodate new values. The performance impact of this algorithm is O(1) until the values start to spill over[12].

## Callback Command Pattern

`with Pace`

Provides a means to attach a class-wide dispatching callback to a command. The callback is contained as a Channel component to the message.

```
-- Server
type Msg_Name is new Pace.Msg with
    record
        Callback : Pace.Channel_Msg;
    end record;
procedure Input (Obj : in Msg_Name);

procedure Input (Obj : in Msg_Name) is
    use type Pace.Channel_Msg;
begin
    -- respond on callback
    Pace.Socket.Send (+Obj.Callback);
end Input;

-- Client
declare
   CB : My_Callback;  -- This is a locally derived Pace.Msg
   Msg : Msg_Name;
begin
   Msg.Callback := Pace.To_Callback(CB);
   Pace.Socket.Send (Msg);
```

```
end;
```

Note that no data components can be attached to the callback message because the server can only see the anonymous wrapper; the Channel_Msg "+" operator does the dereferencing on the response callback. If you wish to add data or simplify the representation, adopt the Notify pattern and think in terms of synchronized "in out" message passing. Or use the Publish Subscribe pattern if you need multiple responses. In fact, the actual reason we included this pattern here is because it essentially falls out of the command pattern and no extra library code is involved; i.e. it's a freebie.

## Persistent Command Pattern

```
with Pace.Persistent
```

Provides a means to save commands to a persistent storage device, i.e. disk.

```
procedure Input (Obj : in Msg_Name) is
    Copy : Msg_Name;
begin
    Pace.Persistent.Put (Obj);  -- Save to the Data Store
    Pace.Persistent.Get (Copy); -- Retrieve from the Data Store
    -- Post-Condition : Copy = Obj
end Input;
```

The command message is saved in a file which has the same name as the full external tag name of the command message.

## Fault Pattern

```
with Pace.Fault
```

Provides a means to store a fault to a queue for later retrieval by a "fault manager" task. Differs from the Log Exception pattern in that a character string name is attached to a command and that command is logged, instead of just logging an exception string.

```
procedure Stop_Fault is -- Client Task
    function Where is new Pace.Log.Unit_ID;
    Msg : Pace.Fault.Id;
begin
    Msg.Name := Pace.Fault.To_Name (Where);
    Pace.Socket.Send (Msg);
end;

declare  -- Fault Manager Task
   Msg : Pace.Fault.ID;
begin
```

```
   Pace.Socket.Send_Out (Msg);
   Pace.Log.Put_Line ("Fault = " & Pace.Fault.To_Str (Msg.Name));
end;
```

## External Stream Representation Pattern

`with Pace.Stream`

Sometimes we may want to override the built-in marshalling (via Ada.Streams) of the Proxy pattern. We can either compress the data by using block binary transfer, thus overriding the standard byte protocol. Or we can create an ASCII text representation of the stream data to interface to external programs, which may not know all the internal Ada data representations (such as floating point representation).

```
type Status is new Pace.Msg with
   record
      Data : ...;
   end record;
procedure Input (Obj : in Status);

package Fast is new Pace.Stream.Binary (Status);
-- Input is inherited.
```

Use the binary pattern with caution since it will not reconstruct controlled elements or XDR representations.

The text pattern is more involved; check the `Pace.Stream` package specification to format the message according to the Ada Tag header protocol.

## Peek/Poke Pattern

(any package body)

Placing a detectable variable in a body. This works best with singleton objects.

```
package body Singleton is

   Variable : Float;
   for Variable'Size use 32;

   ...

   -- Code that modifies Variable (check SGI)

end Singleton;
```

One can also place these variables in a package spec, preferably in a private section so they do not become globally visible. The P4 utilities work by reading the executable's symbol table and matching the addresses.

### Elaboration Order Pattern

(any package spec)

Add a `pragma Elaborate_Body;` in every package specification in the application layer. This prevents circular dependencies in elaboration order and also enforces a layering in the architecture. Avoid placing 'with' dependencies in package spec. Extraneous withing increases compilation time dramatically.

In practice, there are no elaboration order restrictions apart from what was needed by the discrete event simulator, which required the main procedure to be invoked before the simulator could start the clock. Since this is good to do anyways, it was not much of a restriction.

### Execute Once Pattern

(any package body)

Executes a functional operation only once, even if called multiple times.

```
protected Once is
   procedure Operation;
private
   Occurred : Boolean := False;
end Once;

protected body Once is
   procedure Operation is
   begin
      if not Occurred then
         --
         -- Fill in your code here
         --
         Occurred := True; -- Protected from additional attempts
     end if;
   end Operation;
end Once;
```

This can be used in conjuction with elaboration or tasking code whereby multiple clients may try to initialize a data structure but only the first one to reach that point succeeds.

### Environment Variable Pattern

with Pace;

Gets an environment variable from the shell, and returns a default if not available. The default is important if the desired environment is not available from the shell.

```
Value : constant Integer := Pace.Getenv ("PACE_NODE", 1);
```

Overloaded float, integer, and string versions of `Getenv` are available. Environment variables are fairly useful in PACE, the variable called `PACE` itself is used to establish one or more rooted run-time directories.

## Configuration Data Pattern

```
with Pace.Config
```

The guts of this pattern will be covered in detail in a section related to using the PACE knowledge base, but this gives the basics.

```
procedure Input (Obj : in Msg_Name) is
   Value : Integer:
   use Pace.Config;
begin
   Value := Get_Integer ("connection", Parse (Pace.Tag(Obj)));
exception
   when Not_Found =>
      -- Matching value not found in knowledge base
end;
```

A PACE application uses a knowledgebase instead of a database. The initial data string `connection` is called the functor and the trailing parameters are referred to as the arguments. The internal representation as an Ada-embedded Prolog knowledgebase allows greater flexibility in configuring data as rules. The transaction speed has been measured around 2000 transactions/sec. See Annex B for a grammar guide.

## Multicast Pattern

```
with Pace.Socket.Multicast;
```

This is based on an unreliable protocol and should only be used for transient data, such as video or graphics updates. The Receiver object contains a task that dispatches to the correct command destination. The Sender object is a protected object that guards against reentrancy.

```
package body Test is
   -- loopback example, both sender and receiver in the same context
   RX : Pace.Socket.Multicast.Receiver;
   TX : Pace.Socket.Multicast.Sender;
```

```
      procedure Send_Update (Obj : in Pace.Msg'Class) is
      begin
         Pace.Socket.Multicast.Send (TX, Obj);
      end Send_Update;

   begin
      RX := Pace.Socket.Multicast.Create (Pace.Config.Get
("multicast_address", "dvs"));
      TX := Pace.Socket.Multicast.Create (Pace.Config.Get
("multicast_address", "dvs"));
   exception
      when Pace.Config.Not_Found =>
    Pace.Display ("Multicast Address not found");
   end;
```

Multicast messages should be kept below the MTD size of 1500 Bytes.

Caution: There is currently only a backend implementation for Unix.

## Shared Memory Pattern

```
with Pace.Keyed_Shared_Memory
```

This pattern was developed for an  Ada 95 Differences class. It effectively demonstrates how a O/S specific interface can be abstracted away to look like a simple memory access.

```
type Data_Type is
  record
     Int : Integer := 0;
     Flt : Float := 0.0;
  end record;
type Data_Block is access Data_Type;

Pool : Pace.Keyed_Shared_Memory.Block (Key => 700);
for Data_Block'Storage_Pool use Pool;

-- To access memory, dereference the instance
Value : Data := new Data_Type;
```

The `Key` and `Size` values need to be identical across applications.

Caution: There is currently only a backend implementation for Unix.

## 6DOF Pattern

```
with Pace.Socket.Multicast;
```

A concrete example of a command message used to interface to an external 3D visualization tool via the multicast pattern. The tool we currently use is ProE dvsMockup.

```
      use Ada.Strings.Unbounded;

      -- Server Side
      type Position is new Pace.Msg with
          record
              Assembly : Unbounded_String;
              X, Y, Z : Float;
              A, B, C : Float;
          end record;
      procedure Input (Obj : in Position);
      -- body implements the graphics update

      -- Client Side
      TX : Pace.Socket.Multicast.Sender;

      declare
         Msg : Position;
      begin
         Msg.Assembly := To_Unbounded_String ("gyrator");
         Msg.X := 0.0;
         Msg.Y := 0.0;
         Msg.Z := 0.0;
         Msg.A := 0.0;
         Msg.B := 0.0;
         Msg.C := 0.0;
         Pace.Socket.Multicast.Send (TX, Msg);
      end;

   begin
      TX := Pace.Socket.Multicast.Create (Pace.Config.Get
("multicast_address", "dvs"));
   exception
      when Pace.Config.Not_Found =>
    Pace.Display ("Multicast Address not found");
   end;
```

This example gives the flavor of the pattern. Typically, we may abstract the client-side send into a library-level service call, since many concurrently executing objects will update the graphics server simultaneously.

## CommandLine Pattern

```
with Ada.Command_Line;
```

Variation of the Environment Variable pattern applied to the Command_Line.

Summarizing the list of patterns, several observations stand out:

1. No where are timeouts (i.e. abort processing) specified
2. No where are priorities specified
3. Rarely are the Ada `new`, `access`, or `alias` keywords used.

In general specifying timeouts and priorities early on in prototyping is a a recipe for disaster. Timeouts in code lead to extremely non-deterministic behavior and is a relatively useless mechanism when the default behavior is to retry anyway. Use the `Pace.Log.Timer` services if critical timing requirements are needed. Priorities have much the same impact and add little value early on when the discrete event simulator plays a bigger role in determining execution robustness.

The lack of pointers in the library is intended. For data structures, we depend more on stack space and controlled abstractions to manage memory. This eliminates the potential for dangling pointers and memory leaks.

**Server Instancing**

Other container abstractions that can be included in the set of patterns include lists, ring buffers, and specialized server task applications. We keep the latter separate from the other patterns, in that they are not typically used repeatedly in the code, at least not enough to warrant being called a pattern.

WebServer | P4Parser | LogicalNode | KnowledgeBase

### Web Server Instancing

```
with Pace.Server.Home;
```

Instancing a multi-tasking web server in the main procedure is needed to use the URL Command pattern. The number of reader tasks is configurable.

```
   Pace.Server.Home.Create (Number_Of_Readers => 3,
Storage_Size_Per_Reader => 100_000;
```

An embedded web server works very effectively as a stimulator. The URL command pattern provides back-door stimulus to code behaviors. The intent is that prototyped GUI interface code to the PACE application can then be invoked through a browser.

Web ports are set automatically starting from 5600 and incremented by adding the node number to 5600. As an example, PACE_NODE=3 would force the server tasks to listen on port 5603.

## P4 Instancing

```
with Ses.PP, Text_IO;
```

Instancing a Peek/Poke server in the main procedure is needed to use the P4 test utilities.

```
   -- ... with packages
   procedure Main is
   begin
      -- ... other calls
      Ses.PP.Parser;
      -- will never get here
   exception
      when Text_IO.End_Error =>
         Ses.Os_Exit (0);  -- exits if ^D entered
   end Main;
```

This takes over the thread of the main procedure. More information on using this test environment is in Annex C.

## Rule Process Instancing and Use

```
with Gnu.Rule_Process;
```

Instancing a new Rule Process may be useful for other applications. We follow the guts of the `Pace.Config` package body to see how this is done. The object `KB` is the embedded knowledgebase server; it is given a default stack storage space of 100,000 bytes.

```
package body Pace.Config is
    use Gnu.Rule_Process;

    KB : Agent_Type (100_000);

    function Get_String (Name : in String; Id : in String) return
String is
        V : Variables (1..2);
    begin
        V(1) := +Id;
        KB.Query (Name, V);
        return +V(2);
```

```
    exception
        when No_Match =>
            raise Not_Found;
    end Get_String;

    procedure Load (File_Path : in String) is
    begin
        KB.Load (File_Path);
    exception
        when No_Match =>
            Pace.Error ("File Not Loaded " & File_Path);
    end Load;

    Pace_Path : String := Pace.Getenv ("PACE", ".") & "/";
    begin
      for I in 1 .. Gnu.Fields.Count_Fields (Pace_Path, ':') loop
         declare
            File : String := Gnu.Fields.Select_Field (Pace_Path, I,
':') & File_Name;
         begin
            if File_Exists (File) then
               return File;
            end if;
         end;
      end loop;
      return "";
   end Find_File;

begin
    KB.Init (Ini_File => "",
             Console => False,
             Screen => Pace.Getenv ("PACE_GRP_DEBUG", 0) = 1,
             Ini => (Clause    => 1000,
                     Hash      => 507,
                     In_Toks   => 500,
                     Out_Toks  => 500,
                     Frames    => 4000,
                     Goals     => 6000,
                     Subgoals  => 300,
                     Trail     => 5000,
                     Control   => 700));
    -- Initial File Load
    Load (Find_File ("/config.pro"));
end Pace.Config;
```

Queries such as that used in `Get_String` are used to retrieve information from the server. The direct query

```
   KB.Query (Name, V);
```

asks if there are any matching functors starting with the lower case `Name` and followed by the variable list `V`. The first argument in the variable

list `V(1)` happens to be bound to the value `Id`. The unbound argument `V(2)` returns the matched knowledgebase value. If a value was not found, then the exception `No_Match` is raised.

The complement of a query is an assertion. An assertion can be formulated:

```
declare
   Fact : constant String := F("switch", "power"+"off");
                              -- "switch(power, off)."
Begin
   KB.Assert (Fact).
End;
```

The `Assert` call always succeeds. Once a fact is asserted, it can be retrieved through the query:

```
declare
   V : Variables (1..2);
begin
   V(1) := +"power";
   KB.Query ("switch", V);  -- converts to "switch (power, V2)?"
   Text_IO.Put_Line (+V(2));
end;
```

The `Ini` defaults can be adjusted depending on how much memory needs to be reserved for resolution of queries and storage of rules and data.

## Logical Node Instance

`(any executable)`

An environment variable called **PACE_NODE** takes on an integer value which is used to distinguish between executing simulations. The nodes are typically numbered starting with **PACE_NODE**=1. The socket-based communication patterns use this node numbering scheme to determine routes.
If **PACE_NODE**=0, then the executable is stand-alone and it is routed internally.

# PACE How-To

All Inter-Process Communication (IPC) in PACE occurs through a message-passing paradigm built on top of the Ada "**command**" pattern. This pattern requires that you construct anticipated data communication proxies by inheriting from the `Pace.Msg` data type.

In addition to the command pattern, the other fundamental concept behind PACE communications is the structuring of executables into logical nodes. If you use the P4 test environment (Annex C) already, you will be familiar with how nodes numbered 1..N get associated with executing processes. PACE uses the same node numbering scheme to identify communication ports. Similar to the `session.pro` file used in P4, you will use the user-created `nodes.pro` file to define communication "**connection**" paths. The basic premise is that the PACE "**command**" messages are routed through "**connection**" paths at run-time.

### Ada Implementation

The rules for implementating the PACE command messages are simple and few. For the server side of things, add your derived messages to a package spec as follows (the full code for this example is shown in Annex H):

```
--
-- Server-side Command Service
--
with Pace;
package Gyrator is
    pragma Elaborate_Body; -- Singleton Object

    type Status_Type is (Halted, Moving);

    --
    -- Inputting a Move msg will start a motion
    type Move is new Pace.Msg with null record;
    procedure Input (Obj : in Move);
    -- Semantics: Guarded/Protected, Blocking

    --
    -- Requesting a Get_Status will indicate if in motion
    type Get_Status is new Pace.Msg with
       record
```

```
            Value : Status_Type;
         end record;
      procedure Output (Obj : out Get_Status);
      -- Semantics: Guarded/Protected, Non-Blocking

      --
      -- Inputting a Halt msg will stop a motion
      type Halt is new Pace.Msg with null record;
      procedure Input (Obj : in Halt);
      -- Semantics: Guarded/Protected, Blocking

   end Gyrator;
```

Note that each message must be associated with either an `Input`, `Output`, or `InOut` command pattern primitive operation. The names of these primitives are meant to mimic the "in", "out", or "in out" semantics of the associated Ada parameter passing mechanism.

Specifying such a package becomes the basis for performing IPC calls. To make these available for remote IPC dispatching, *somewhere* in your code you will need to "with" the `Pace.Socket` package. That's it for implementing the server.

Implementing the client is even easier. Inspect the command message you wish to use, and then choose one of the three `Pace.Socket` services to match the semantics:

- `Pace.Socket.Send` (for Input messages)
- `Pace.Socket.Send_Out` (for Output messages)
- `Pace.Socket.Send_InOut` (for InOut messages)

The Send command has an additional `Ack` argument that allows you to switch to "send and forget" semantics. This essentially changes the behavior from synchronous (Ack=TRUE) to asynchronous (Ack=FALSE).

As an example of client sending to server:

```
with Pace.Socket;
...
procedure Call_Move is
   Move_The_Gyrator : Gyrator.Move;
begin
   -- Send a Move command
   Pace.Socket.Send (Move_The_Gyrator);
   -- Received ack on the sent Move command
end Call_Move;
```

That's it for implementation.

## Run-Time Configurations

The construction of the run-time file `nodes.pro` is also straightforward. First determine a "host_node" sequence of integers consecutively running from 1, 2, etc. For example, the client could be node 1 and the server node 2. Use the same numbering scheme as you would for P4 to eliminate any potential confusion if at all possible.

The names of the command messages are **lower case** versions of the full Ada name[13] with 'dot' punctuation converted to commas. So "GYRATOR.MOVE" becomes "gyrator,move".

Create the `nodes.pro` file using a text editor, in a fashion similar to how the P4 `session.pro` file gets created:

```
%%%% This is a 'nodes.pro' file.
%%%% The '%' character works as a comment delimiter.
connection (gyrator, move, 2).
connection (gyrator, get_status, 2).
connection (gyrator, halt, 2).

host_node (1, "bin39-04-1").
host_node (2, "bin39-05-1").  %% 'gyrator' messages sent to this node
```

Because the command messages are broken down into their constituent components via the comma delimiters, we can use the wild card match character '_' to send all messages to the same server logical node:

```
%%%% Simpler 'nodes.pro' file.
connection (gyrator, _, 2).   %% All operations in gyrator sent to Node 2

host_node (1, "bin39-04-1").
host_node (2, "bin39-05-1").  %% 'gyrator' messages sent to this node
```

To actually attach the node number to an executing process, input the following from the UNIX command-line:

```
env PACE_NODE=1 client_exec_name
```

However, keeping in the spirit of P4, you can instead add the `PACE_NODE` environment variable to the `session.pro` file[14]:

```
proc (1, "client_exec_name", env("PACE_NODE=1"), arg("")).
```

## Implementation Notes

1. Ports are set starting from 5500 and incremented by adding the node number to 5500. So Node=3 would be at port 5503.
2. The PACE socket services does not have the problem of closed sockets causing the user to wait for several minutes before attempting to use the same port after a test run. This works because PACE configures the port for reuse.
3. Stubs on the client side are created by taking the original package spec and commanding GNATSTUB to generate a body. That's it.
4. No limit to the size of the messages. Command messages can grow to the size of the stack
5. All local calls are automatically short circuiting. If you would like to test an executable in a purely local node, then set either the host node to 0, remove the `nodes.pro` , or enter 0 in the `nodes.pro` file for all destinations.
6. For concurrently executing Asynch calls running in **local** mode, a surrogate task is created to hold the executing thread. This is required to maintain the same semantics as the remotely executed call.
7. Messages containing components constructed from the heap can be sent over the network **if** they are based on `Controlled` types.
8. A Name Server approach for generating port numbers is also available and the hooks are already in place. If needed, instructions for how to configure the Web-based Name Server will be added to this document.
9. Select a unigue PACE_NODE for each executable meant to communicate across IPC, use the P4 node for convenience.

Environment variables

| Variable Name | Type | Default | Description |
|---|---|---|---|
| PACE | PathNames | "." | Directory Names for run-time files seperated by colons (similar to $PATH) |
| PACE_NODE | Integer | 0 | Logical node number |
| PACE_PORT_MSG | Integer | 5500 | Starting Messaging Port number of Node=0 |
| PACE_PORT_WEB | Integer | 5600 | Starting Web Port number of Node=0 |
| PACE_NODES | FileName | nodes.pro | Nodes file |
| PACE_TASK_STACK | Integer | 30_000 | Size of receiver task stack in bytes |
| PACE_MAX_SURROGATES | Integer | 10 | Number of async messages allowed concurrently |
| PACE_INIT | 1 or 0 | 1 | 1 turns on auto-elaboration of Pace.Socket services |
| PACE_SERVER | Name | "" | Name Server Host:Port |
| PACE_HOST | Name | "localhost" | Local Host Name |

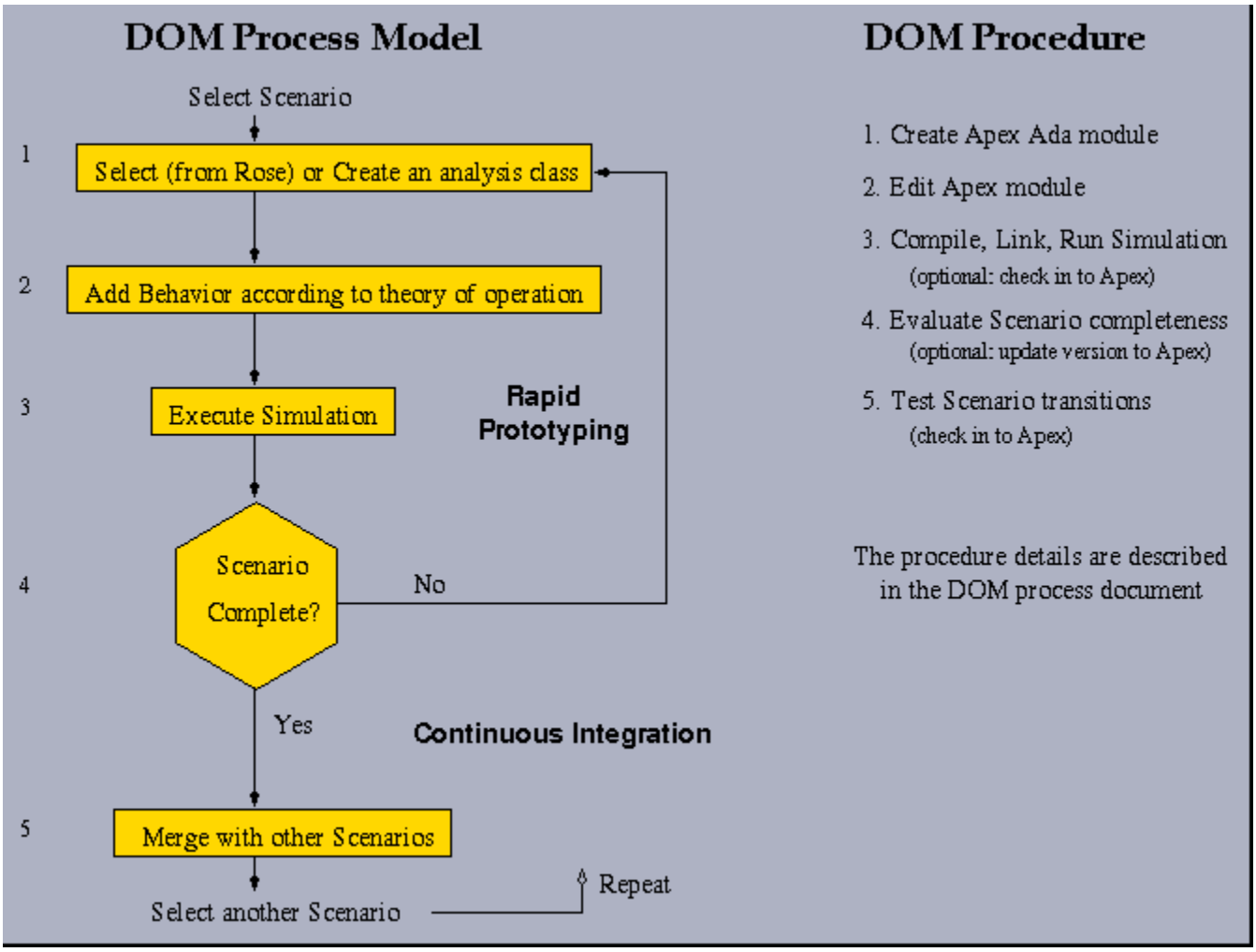
| | | | |
|---|---|---|---|
| PACE_DISPLAY | 1 or 0 | 1 | 1 turns on Pace internal display messages |
| PACE_DISPLAY_DEBUG | 1 or 0 | 0 | 1 turns on Pace.Log.Trace debug messages |
| PACE_TRACE_OUT | FileName | "trace.out" | Trace Output file (set to "" if no trace desired) |
| PACE_SIM | 1 or 0 | 0 | 1 turns on Discrete Event Simulation mode |
| PACE_TIME_SCALE | Integer | 1 | Time Slowdown factor, if negative then Speedup factor |
| PACE_RUN_TIME | Float | 0.0 | Total Run Time before exiting, if 0.0 then infinite |
| PACE_LATENCY | Float | 0.0 | Adjustment factor for real-time runs to enable them to match simulation runs |
| PACE_NO_WAIT | 1 or 0 | 0 | Setting to 1 removes all wait times |
| PACE_SIM_TICS | Integer | 1 | Slows down discrete event simulation if > 0 |
| PACE_GRP_DEBUG | 1 or 0 | 0 | Knowledge base display debug |

# Extreme Programming Rationale

This section captures the rationale for what is called the "Extreme Programming" development process. The practice then was to use a common branch of source code and perform continuous development and integration with several developers simultaneously involved. (Even though source is shared, each developer's executable is unique because the object/link area is separate from the source directory.)

In some circles this is called "hacking" but it seemed to work for us, and it kept maintenance costs way down due to the minimal set of sources required to be managed. The complete process document that we used for guidance weighed in at short 5 pages.

The figure below showed the general flowchart for developing code in terms of a hypothetical dynamic object (or agent) model (DOM). At all stages we were collecting visual artifacts (operator screens, solid model sims, directed graphs, etc)




A few of the lessons learned from development that should adapt to PACE include:

1. Place stub behavior in code even though it won't be used in the short term. This is the basis for all iterative and lo-fi development.
2. The Autocode generation that the DOM used was barebones. Code generation in general gives you the easy part, the hard part involves working the behaviors. Cue in a LLM. This is important to keep in mind as the behaviors in the past could not be generated automatically nor could they easily be inherited.
3. We did not overly rely on debuggers, as all debuggers will cause interactive multitasking programs to halt. Instead the interactive P4 can be used.
4. The DOM has no dependence on task priorities. This probably helps us eliminate lots of subtle bugs and to freely add tasks as needed[15].
5. GNAT is a compiler that is not based on the traditional compilation library. This means that there are very few mysterious rebuilds and that it is very easy to tell if some code has been changed.

6. Monitor thread stack sizes carefully and use the compile switch that monitors stack overflow. The DOM has a mechanism to adjust the thread stack sizes uniformly or individually; see the PACE pattern on naming Agents.
7. Use the trace facility.
8. If tasks are blocked or things just stop, watch out for overloaded Input calls to task entries within a task. This will cause deadlocking. Use the dot (.) qualified full notation to make the correct call, instead of relying on a "use" clause (which may grab the innermost scope).
9. We have had good success with the `module` facility[16]. Use the facility to load the necessary environment variables; e.g. add the following lines to your `~/.personal` file:
   ```
   use /directory_path/modules
   module add pace
   ```
10. **Changing main driver.** Customize or add top-level behavior to the executable by either editing the default main driver or by modifying a copy of the driver. If the latter, the new driver must be renamed. The startup code should include `Pace.Log.Agent_Id` to register the main procedure.
11. **Stimulating the driver** The executable is stimulated through main drivers (statically linked or communicating through IPC) or via the stimulus web page.
12. **Creating DOM source code files.** Add main procedural drivers into the `drivers` directory. Add packages to the `src` directory.

---

# Summary

1. Toward device independent (portable) software. For example, in future systems where hardware and middleware specifications are not yet defined, it is beneficial to develop software independent of the target hardware.
2. Build a business case for doing and using the above. Software is very expensive. The PACE library should be a part of a cradle to grave software lifecycle paradigm (ideally, we minimize what we throw-out and redo in software and drive the software prototype right on to the objective system). Result - competitive advantage (decreased time to market and cost, increase in product quality).

**Key Features**:

1. DOM and PACE architecture kernel are hardware independent and portable.
2. These technologies can be used to develop the system from the conceptualization phase, through the actual rollout on the objective system cradle to grave.

3. The architecture and kernel are totally technologically current: Built on the object oriented paradigm, fits seemlessly into a UML development model, can be ported to any object oriented computer language (currently in Ada, could move to Java, C++,…), fully incorporates the use of Web technologies, and has a reusable / adaptable communications library capable of supporting any point to point or broadcast communications paradigm.

# Annexes

THE ANNEXES

PACE API Reference

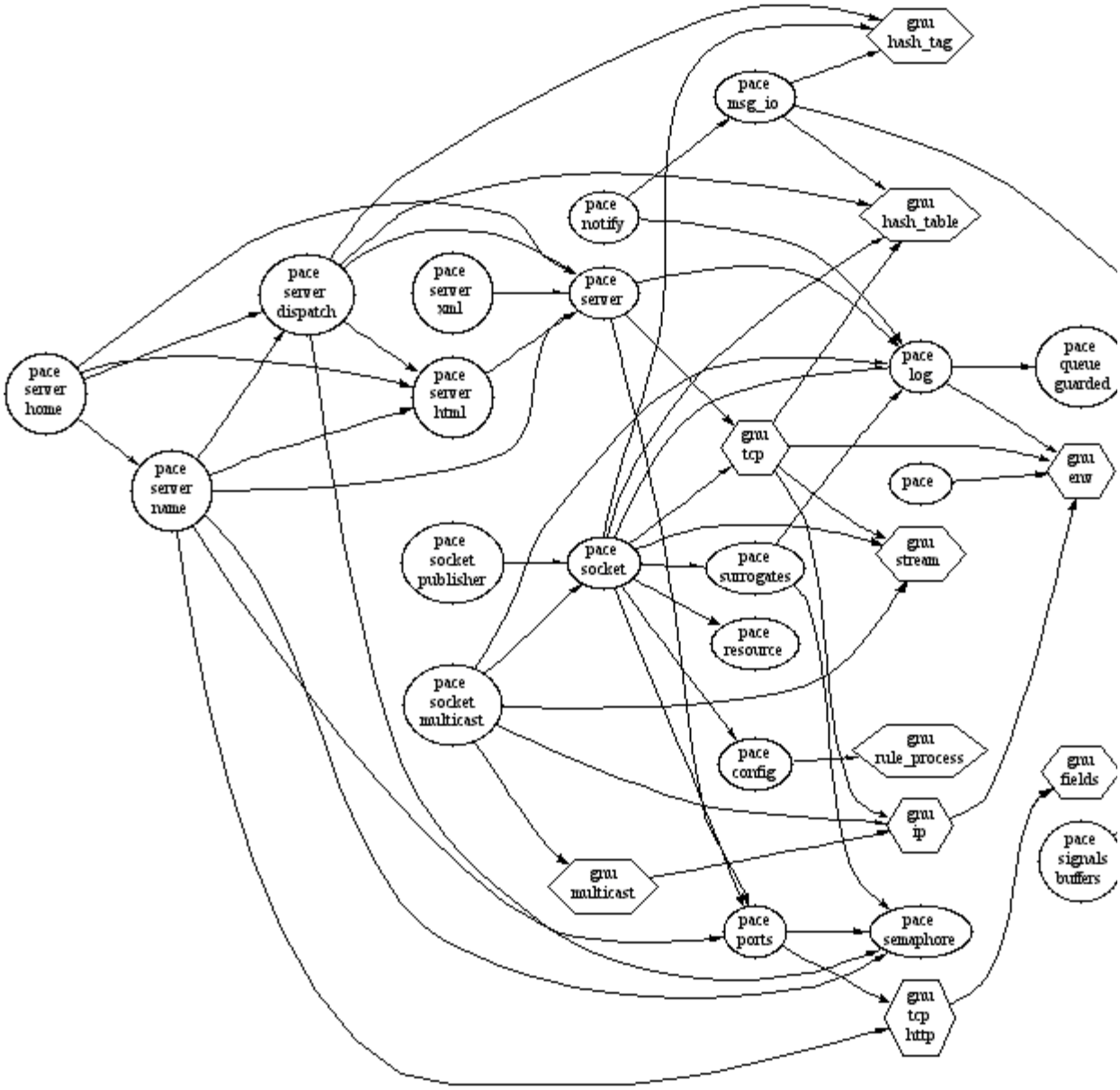
A. Gnu.Rule_Process/Prolog Reference
B. P4 Reference
C. Ada Training Reference
D. DOM Process Reference
E. Discrete Event Simulation Rationale
F. RTCOE Review
G. PACE Example Source Code ("The Gyrator")
H. Bibliographic References
I. Footnotes

---

## Annex A

**PACE Application Programmer Interface Reference**

PACE interface | Gnu interface




***PACE Library Mutual Dependencies*** : Dependencies to the PACE root package are not shown to simplify the representation.

# Annex B

## Gnu.Rule_Process (Prolog) Reference

The `Pace.Config` package makes calls to a GNU library package called `Gnu.Rule_Process`, which provides querying and assertion services to a run-time inference engine. The inference engine was designed to use the backward-chaining semantics and first-order logic syntax of Prolog[17]. The potential for combining logical

rules and facts (i.e. data) in Prolog enables us to provide a very flexible back-end to configuration data. This concept is commonally referred to as a knowledgebase, in that it combines the strengths of a rulebase with a regular database. When it is used in an application, the inference engine plus the knowledgebase is often referred to as an Expert System. So in the case of PACE IPC communications, the basic need accomodated is in providing an expert system to determine routes between PACE nodes.

An elementary introduction to Rule_Process Prolog-syntax is here. The advantage of Prolog is that it is simple to represent in terms of an ASCII text file and one can maintain a relatively uniform representation without the need for deep levels of nesting (ala LISP). (For examples of how the Prolog is being used in a test suite, browse through examples of the P4 `session.pro` files, see Appendex B.)

The front-end interface to the back-end engine is provided by the `Gnu.Rule_Process.Agent_Type` class. The `Agent_Type` class encapsulates a task with entry points which provides a reentrant safe mechanism to query rules and facts and dynamically load a knowledgebase. A class can be instanced multiple times if there is a need to keep unique knowledgebases separated.

# Annex C

### Peek/Poke/Performance Profiling (P4) Reference

Go to <placeholder> for documentation on the P4 testing approach.

# Annex D

### Ada Materials

<placeholder>

# Annex E

### DOM Reference Material

<placeholder>

# Annex F

### Discrete Event Concurrent Simulation

The specification of concurrent distributed systems is hard (what is known in computer science circles as NP-complete). If done ambiguously, any understanding of the specification by a knowledgeable individual is even harder to come by. In particular, we (as software engineers) are routinely subjected to bubble diagrams of another engineer's understanding of the internal workings of a hypothetical system. However, without a formal basis and nomenclature, these diagrams are nearly worthless and equivalent to the back-of-the-envelope sketches worked out in brainstorming sessions. Its no wonder that the testing guru Robert Binder in his book "Testing Object-Oriented Systems" [Binder 2000] calls such sketches cartoons and states that the testing of software via such artifacts becomes extremely problematic.

In our opinion, handing over sketches to a group of software designers and developers as requirements documents is equivalent to presenting a Rubik's Cube and then saying "Solve this!"

Well then, what is wrong with the diagrams? The fundamental goal of a good SW requirements specification is to present information that moves the development forward and away from "analysis paralysis" but not leading to chaos and/or "gold-plating". Anything less than this only serves to distort or confuse the developer (causing slip or cost overruns in the finished product). That is the essential issue with all diagrams being created; the lack of automated checking (either through simulation or theorem checkers) causes the developers to spend unnecessary time questioning every aspect of a diagram's flow. As most of the model-based requirements literature states, a degree of formality is essential for getting past the limitations of any diagram generated. Anything less than this defers the problem to another person down the line and ends up costing more in the long run or worse yet causing a vicious cycle of diagramming exercises followed by rework.

The problem can be obviated by providing both a formal model (spec) and simulation (verification) as a means to get it right the first time (because there never will be a next time due to cost). Without a formal, or semi-formal, and universally understood nomenclature that translates problem scope and solution from high levels of abstraction to detailed levels close to implementation, the path forward becomes intractable and only *ad hoc* approaches remain.

# Annex H

### Gyrator Example

For educational purposes, we provide a somewhat contrived example that whatever it's shortcomings in modeling a real-world application, it at least walks through many

PACE patterns[19]. This is an elaboration of the Proxy example provided in the PACE How-To section.

The general idea is that we have two remote applications, a server that abstracts an electro-mechanical device called a "gyrator" and a client that controls the device designated as the "controller". Importantly, note that both of the applications are best described as singleton classes[20]; only one "gyrator" exists in the system and only one "controller" exists in the system. If, as we add more functionality, there came a need for adding another controller for a different device, that controller singleton would need a different name (e.g. "controller_x") to distinguish it from the gyrator's controller singleton.

| Gyrator Files | |
|---|---|
| **Client Main** | **Server Main** |
| Controller Spec | Gyrator Spec |
| Controller Body | Gyrator Body |

Three config files govern the run-time aspects of the executables.

- The main config file (config.pro)
- The node/routing table (nodes.pro)
- The launching session (session.pro)

# Annex I

**References**


[Schmidt 1994]
D. Schmidt, "The ADAPTIVE Communication Environment: Object-Oriented Network Programming Components for Developing Client/Server Applications", 12th Sun Users Group Conference, June 1994.

[Kamrad 1998]
M.Kamrad (revised/presented by P.Pukite), "Ada 95 Differences Training", 1998.

[Pukite 1992]
P.Pukite et al, "Expert System for Redundancy and Reconfiguration Management", Proceedings of the IEEE National Aerospace and Electronics Conference, (IEEE, NY, 1992), p.233.

[Pukite 1998]
"Markov Modeling for Reliability Analysis", Wiley-IEEE Press, 1998.

[Pukite 1993]

"Automated Interface Code Generation from Ada Specifications", ACM Ada Letters, 13/3 (1993) p.74.
[Gamma 1994]
E.Gamma et al, "Design Patterns: Elements of Reusable Object-Oriented Software", Addison Wesley, 1994.
[Sanden 1997]
B.Sanden, "Modeling Concurrent Software", IEEE Software, 1997, p.93.
[Booch 1996]
G.Booch, "Managing Object-Oriented Software Development", 1996.
[Binder 2000]
R.Binder, "Testing Object-Oriented Systems", 2001.
[Hovel 1995]
D.Hovel, "Using Prolog in Windows NT Network Configuration", Proceedings of the Practical Applications in Prolog Conference, 1995.
[Wood 1992]
D.P.Wood and K.C.Kang, "A Classification and Bibliography of Software Prototyping", CMU/SEI-92-TR-13, 1992 .

# Annex J

**Footnotes**

1. (empty)

2. The term *GoF* referes to the "Gang of Four" authors who wrote the patterns textbook.

3. Often O/S library layers show a reliance on *GoF* C++ patterns which were re-implemented in Ada.

4. As a counter example, observed instances where embedded code used more than 8000 semaphores because the advantages of pooling or dedication of OS semaphore resources among or to tasks was not incorporated.

5. In real time a test run consumes upwards of an hour of wall-clock time. Simulation would reduce that run to a few seconds, and the problem would be resolved quickly.

6. Note that a real-time environment may have a concept of simulated clock and potentially could do a form of discrete event simulation, but this will likely have to be worked in from the base layers.

7. Because of portability requirements to be able to run on Windows-based stations, the non-portable UNIX IPC constructs such as shared memory and message queues were early abandoned in favor of the more universal socket layer. The socket mechanisms mesh well with the command pattern and Ada streams, as well as embedded web servers.

8. Remember the old adage postulating that compile and link times are the bane of statically compiled languages? GNAT is a fast compiler.

9. A lesson learned is not to rely on a Booch/UML model or code generation from a model. Because the DOM patterns were simple enough and the singleton patterns expressible though Ada PDL (Program Description Language), it actually caused excessive work to go through what we called a "Rose Update" compliance check. Executable Architecture's early failure to germinate was likely due to the overly strict requirement for use of "tactical" software, not to mention the extensive set of process steps required to convert from a complex Rose model to a set of Ada packages that would execute out-of-the-box.

10. Automotive protoptyping indicate that tiered architectures are being used by more and more systems. As the lifecycle progresses there is concern on how this paradigm will be deployed for real-time systems.

11. The command pattern on top of sockets is efficient. TCP/IP works very well. We never considered going with UDP or multi-cast because of the potential for dropped messages. When dropped messages are not important, TCP/IP is still a good choice as the current literature gives it the same or better performance than UDP. For TCP/IP, one must also disable the Nagle algorithm. This is a pretty obscure fact so it qualifies as a bona-fide trick.

12. Software optimized from O(N) search time to O(1) by incorporating hash tables. This was needed as the system integration of software revealed significant slowdowns in certain list processing algorithms.

13. This is equivalent to `Ada.Tags.Expanded_Name`

14. We will easily be able to absorb the PACE_NODE declaration into the P4 launching process, and make this the same as the PID value.

15. The DOM code does not rely on task priorities except for a low priority task when running in Simulation mode. This requires a task library that strictly enforces priorities and does not allow tasks to drift into a "time-sharing" roles if they get

starved. We observe that certain native thread libraries such as Solaris 5.6 will bump priorities but not Solaris FSU threads.

16. This allows the paths and environment variables to easily be modified. See the **module** man pages.

17. The use of Prolog to provide rules has actually been used in such commercial applications as the Windows NT Network Configuration [Hovel 95]. In practical terms, the need that PACE applications have in setting up Socket routing connections is much the same as an large network.

18. The GNAT Ada compiler has two on-line sources: the GNAT User's Guide ("gnat_ug.html") and the GNAT Reference Manual ("gnat_rm.html"). For details on making contolled views refer to Apex via the Online Help. Use the Ada Reference Manual to resolve language issues.

19.Over 70% of the PACE code is reused from P4 and 100% of the PACE code has been used in DOM simulation development.

20.Singleton classes (i.e. singly instanced packages) are perfectly fine to use.